

TrivialTier: 基于属性的 SD-WAN 控制系统设计

从声明式策略到原生态网络执行的演进

李梓勤、周杰
指导教师：高凯

四川大学网络空间安全学院

2025 年 12 月 23 日

目录

- 1 研究现状
- 2 我的工作
- 3 属性模型
- 4 核心逻辑
- 5 演示验证

研究动机：SD-WAN 的语义缺失

- **现状：**现有的个人/SOHO SD-WAN (如 Tailscale) 策略多基于静态 Tag 或 IP。
- **局限性：**
 - 无法感知动态环境 (如威胁等级、地理位置)。
 - 缺乏对 L4 细粒度协议的声明式描述。
 - 策略表达力 (Expressiveness) 受限，难以适配零信任模型。
- **目标：**构建一个逻辑集中化的控制平面，实现属性驱动的自动化安全部署。

TrivialTier 系统概述

核心功能

- 设计并实现了基于 ABAC 的 SD-WAN 策略引擎
- 支持声明式策略定义，自动生成 WireGuard 和 nftables 配置
- 实现属性驱动的动态访问控制，支持主体、客体、环境多维判定

技术特点

- **声明式策略**：通过 YAML 定义安全策略
- **自动化生成**：从策略到配置的自动化编译流程
- **细粒度控制**：支持 L3 (WireGuard) 和 L4 (nftables) 执行

属性模型：主体、客体与环境

主体属性 (Subject)

- `system.os`
- `system.patch_level`
- `network.zone`

客体属性 (Object)

- `identity.role`
- `identity.owner`

环境属性 (Environment)

- `global.threat_level`

示例策略

```
subject: system.patch_level >= 20251101
object: identity.role == "storage"
environment: threat_level < 3
```

核心逻辑：声明式属性判定

```
if operator == "==":
    return actual_value == expected_value
elif operator == "!=":
    return actual_value != expected_value
elif operator == ">":
    return actual_value > expected_value if isinstance(actual_value,
        (int, float)) else False
elif operator == ">=":
    return actual_value >= expected_value if isinstance(actual_value,
        (int, float)) else False
elif operator == "<":
    return actual_value < expected_value if isinstance(actual_value,
        (int, float)) else False
elif operator == "<=":
    return actual_value <= expected_value if isinstance(actual_value,
        (int, float)) else False
elif operator == "in":
    if isinstance(expected_value, list):
        return actual_value in expected_value
    return False
elif operator == "not in":
    if isinstance(expected_value, list):
```

- 递归求值：支持 *AllOf*, *AnyOf*, *NoneOf*。
- 属性投影：通过 Dot-Notation 实现检索。
- 环境感知：将全局变量注入判定流程。

演示验证：策略引擎工作流程

执行流程

- ① 读取节点属性和策略定义
- ② ABAC 引擎评估策略，生成访问矩阵
- ③ 自动化生成 WireGuard 及 nftables 配置

输出结果

demo_output/ 目录包含所有生成的安全配置文件

演示验证：策略评估结果

策略要求

- 主体：Linux 系统，补丁等级 ≥ 20251101
- 客体：角色为 storage
- 环境：威胁等级 < 3

评估结果

- laptop_li: 满足条件，可访问
- nas_core: 补丁等级不足，拦截请求

生成的配置示例

```
# laptop_li.conf (WireGuard)
[Peer]
PublicKey = 9xY2...p8Z1=
AllowedIPs = 10.0.0.100/32

# laptop_li.nft (nftables)
chain inbound {
    ip saddr 10.0.0.1 tcp dport 22 accept
}
```


谢谢！

请各位老师批评指正。